TUTORIAL – 0

Performance Profiling and Optimization using GPROF and PERF

Name: yaswanth kumar

Roll Number:S20240010264

Course: High Performance Computing

OBJECTIVE

The objective of this experiment is to compare the performance of Bubble Sort and Quick Sort using execution time measurement, GPROF profiling, compiler optimizations, and hardware performance analysis.

SOFTWARE USED

- Ubuntu 24.04.4 LTS
- GCC 13.3.0
- GNU GPROF 2.42
- Linux PERF (Unavailable on WSL2)

PROGRAM DESCRIPTION

The program generates approximately 18,250 random temperature values. Two sorting algorithms were implemented:

1. Bubble Sort
2. Quick Sort

The execution time of both algorithms was measured using the clock() function.

COMPILATION COMMANDS

Without Optimization

```
gcc -O0 -pg mysort.c -o mysort
```

With O2 Optimization

```
gcc -O2 -pg mysort.c -o mysort_O2
```

With O3 Optimization

```
gcc -O3 -pg mysort.c -o mysort_O3
```

EXECUTION COMMANDS

./mysort

time ./mysort

gprof mysort gmon.out > myreport.txt

perf stat ./mysort

PROGRAM OUTPUT

Bubble Sort Time : 0.900238 seconds

Quick Sort Time : 0.000000 seconds

EXECUTION TIME

Real Time : 0.904 s

User Time : 0.902 s

System Time : 0.000 s

GPROF ANALYSIS

Function CPU Time

bubbleSort() 84.85 %

swap() 15.15 %

partition() 0.00 %

quickSort() 0.00 %

Hotspot Function:

bubbleSort()

COMPILER OPTIMIZATION RESULTS

O0

Bubble Sort Time : 0.90 seconds

Quick Sort Time : 0.00 seconds

O2

Bubble Sort Time : 0.62 seconds

Quick Sort Time : 0.00 seconds

O3

Bubble Sort Time : 0.61 seconds

Quick Sort Time : 0.00 seconds

Observation:

- O2 reduced execution time compared to O0.
- O3 slightly improved the performance compared to O2.
- Quick Sort remained much faster than Bubble Sort.

PERF ANALYSIS

PERF could not be executed because the system uses the Microsoft WSL2 kernel (6.18.33.2-microsoft-standard-WSL2). The required Linux kernel performance tools were unavailable for this environment.

PERFORMANCE COMPARISON

Metric	Bubble Sort	Quick Sort
Execution Time	0.900238 s	0.000000 s
CPU Cycles	Not Available	Not Available
Instructions	Not Available	Not Available
IPC	Not Available	Not Available
Cache References	Not Available	Not Available
Cache Misses	Not Available	Not Available
Hotspot Function	bubbleSort()	partition()/quickSort()

ANSWERS

1. Which sorting algorithm performed better?

Quick Sort performed better because its average time complexity is O(n log n), whereas Bubble Sort has $O(n^2)$. Therefore, Quick Sort completed the sorting much faster.

2. Which function consumed the maximum execution time?

According to the GPROF report, bubbleSort() consumed approximately 84.85% of the total CPU time.

3. How does execution time change with increasing input size?

As the input size increases, Bubble Sort execution time increases rapidly due to its quadratic time complexity. Quick Sort scales much better because of its O(n log n) complexity.

4. How did compiler optimizations (-O2 and -O3) improve performance?

Compiler optimizations generated more efficient machine code, reducing execution time by optimizing loops and instructions. O2 and O3 both improved performance.

5. Which algorithm would you recommend for large datasets?

Quick Sort is recommended because it is significantly faster and more efficient than Bubble Sort for large datasets.

CONCLUSION

This experiment successfully compared Bubble Sort and Quick Sort using execution time and GPROF profiling. Bubble Sort was identified as the major performance bottleneck, while Quick Sort performed significantly better. Compiler optimizations further reduced execution time. Although PERF analysis could not be completed due to WSL2 kernel limitations, the remaining objectives of the tutorial were successfully achieved.